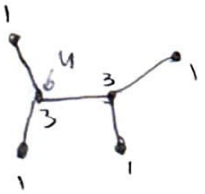
Dylan Trowbridge
CSCI 163/CSEN 179
HW 5

1.

degree $d(u)$ of vertex u is the number of neighbors (edges) u has.



$3+3+4=10$
5 edges

Proof:

The handshaking lemma states $\sum_{u \in V} deg(u) = 2|E|$. This means whether we have an even or odd # of vertices, our ~~total degree~~ will always be even b/c of the definition of evenness. Def. of evenness states if there is some int k so $2k = x$ then x is even.

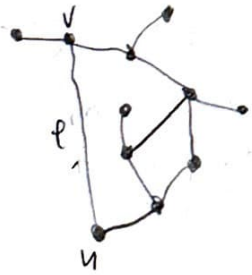
$\sum_{u \in V} deg(u) = 2|E|$ where E is the # of edges (integer) so $\sum_{u \in V} deg(u)$ is even.

$$\sum_{u \in V} deg(u) = \sum_{u \in V_{even}} deg(u) + \sum_{u \in V_{odd}} deg(u)$$

$\sum_{u \in V_{even}} deg(u)$ must be ~~even~~ b/c we add up only even #s.

$\sum_{u \in V_{odd}} deg(u)$ must be odd b/c the sum of $\sum_{u \in V_{even}} deg(u)$ (which is even) and $\sum_{u \in V_{odd}} deg(u)$ is an even number. Therefore, the total # of vertices w/ odd degree in an undirected graph is even.

2.



If e is removed, and u is still marked as visited in the end, then there is ≥ 1 path from v to u and thus a cycle

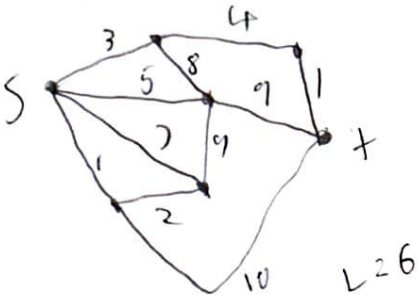
edgeCycleTest($G=(V,E)$, e):
init all vertices as unvisited
init v_1 to be the start vertex of edge e
init u_1 to be the end vertex of edge e
explore(v_1)
if u_1 is visited: return true
else: return false

true means there is cycle

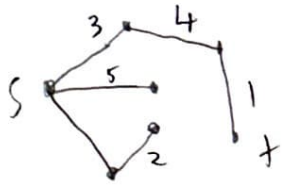
explore(v):
mark v as visited
for every neighbor u of v:
if u is unvisited: explore(u)
return

3. L = gas

$e < L$



$L = 6$



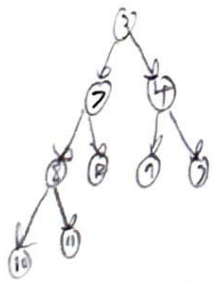

```
cityTravel(G=(V,E), s, t, L):
    for every e ∈ E:
        if e > L: remove e from graph
    init all v ∈ V to be unvisited
    explore(s)
    if t is visited: return true
    else: return false

explore(v):
    mark v as visited
    for every neighbor u of v:
        if u is unvisited: explore(u)
```

First we search the edge set for all edges > L in weight and remove them if they are bigger than L. This is $\Theta(E)$. Then we perform standard DFS which is $\Theta(V+E)$. $\Theta(V+E) + \Theta(E) = \Theta(V+2E)$ which is still linear.

3ec.

The extra credit is just asking for a standard Dijkstra algo pseudocode (using a binary heap). Basic array or linked list yields $\Theta(V^2)$

```
Dijk(G=(V,E), s, t, le):
    Set dist[s] = 0 and dist[v] = ∞ for all other v in G
    init all vertices into priority queue Q w/ insert
    While Q is not empty:
        u = Q.delete-min()
        if u == t:
            return dist[u]
        for every edge uv in G:
            if (dist[u] + luv < dist(v)):
                Q.decrease-key(v, dist[u] + luv)
```



insert: $\Theta(\log(n))$

decrease-key: $\Theta(\log(n))$

delete-min: $\Theta(\log(n))$

we insert V times: $V \cdot \log(V)$

we delete_min V times: $V \cdot \log(V)$

we decrease-key E times: $E \cdot \log(V)$

$\Theta(2V\log(V)) \approx \Theta(V\log(V))$ $\quad \Theta((V+E)\log(V))$

$\Theta(V\log(V)) + \Theta(E\log(V)) =$

Insertion is $\log(n)$ b/c we at most have to do $\log_2(n)$ steps or depth steps. Decrease key is $\log_2(n)$ for the same reason. If we change a value to a lower one, it will at most be swapped around $\log_2(n)$ times. When we delete the min (top node) we again max out at $\log_2(n)$ swaps, but we'll have to take the lower-right most node and put it in its new spot to keep the heap full. This is $\Theta\log(n) + \Theta(1)$ which dominates to $\Theta(\log(n))$

4. cost(G=(V,E), l_e, (v, S);

Set dist(s) = 0 and dist(v) = ∞ for all other v in G and cost[v] = ∞ for all v

dist[s]=0
cost[s]=l_s

init all vertices into priority queue w/ insert

while Q is not empty:

u = Q.delete-min()

for all edges ∈ E:

temp = dist[u] + l_{uv} + l_v

if temp < cost[v]:

cost[v] = temp

return cost[]

5.

G = (V, E)

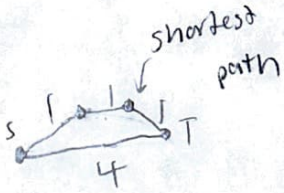
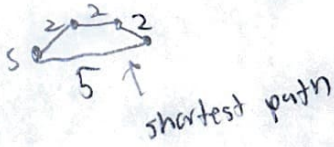
MST

+1

a. MST will not change b/c every edge increases by 1. The smallest edges, the ones making the MST, will remain the smallest

b. However, the shortest paths can change (see example)

ex



+1